MUBD

Màster Universitari en Enginyeria de Dades Massives (Big Data)

Estadística

laSalle ENG
Universitat Ramon Llull

Índice

Fundamentos de clasificación supervisada

01 Introducción a la clasificación

02 K vecinos más próximos (KNN)

03 Naive Bayes

04 Ensambles

05 Recordatorio: validación y métricas

Clasificación

Introducción

01

laSalle ENG
Universitat Ramon Llull

Clasificación

En entrenamiento conocemos los predictores X y la clase y. Aprendemos una regla para predecir la clase de observaciones nuevas.

Tarea	Información disponible	Objetivo
Regresión	Predictores X y respuesta numérica y	Predecir un valor
Clasificación	**Predictores X y etiquetas conocidas y**	**Asignar una clase a un caso nuevo**
Clustering	Predictores X, sin etiquetas objetivo	Descubrir grupos por similitud



Las clases se definen antes de entrenar. Codificarlas como 0, 1 o 2 no convierte la respuesta en una magnitud numérica.

Regresión logística y otros clasificadores

Misma tarea, distintas formas de aprender

La regresión logística también es aprendizaje automático. Estos métodos usan etiquetas conocidas y pueden producir probabilidades y una clase.

Método	Qué aprende o conserva	Cómo predice
Regresión logística	Coeficientes de una función de los predictores	Transforma la función en probabilidades de clase
KNN	Casos etiquetados y una regla de cercanía	Busca vecinos y combina sus etiquetas
Naive Bayes	Probabilidades de clase y de los predictores por clase	Combina probabilidades con el teorema de Bayes

Tipo de respuesta: logística binaria para dos clases, multinomial para varias sin orden y ordinal si hay orden. **KNN y Naive Bayes admiten varias clases, pero no incorporan ese orden automáticamente.**

Clasificación

K vecinos más próximos (KNN)

02

Aprendizaje basado en instancias

KNN: predicción local y efecto de K

Compara cada caso nuevo con ejemplos almacenados y generaliza a partir de su cercanía.

Entrenamiento

Guarda predictores y etiquetas. Puede crear un índice de búsqueda.

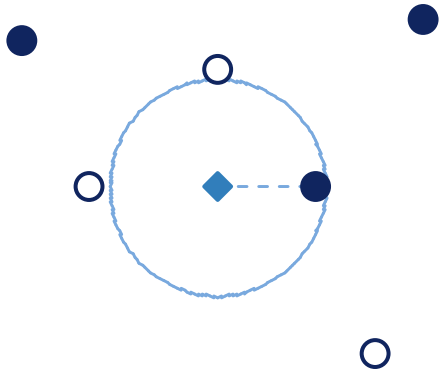
Predicción

Busca K vecinos y vota sus clases para construir una respuesta local.

Más casos exigen más memoria y búsqueda.

Mismos datos. A: relleno. B: vacío. Nuevo: rombo azul.

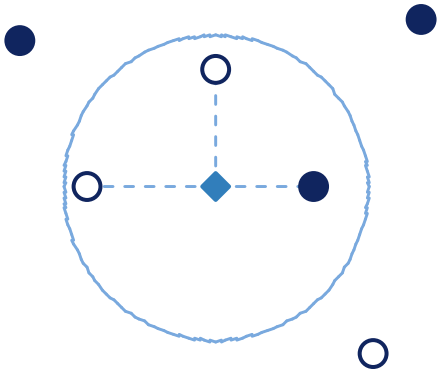
K = 1



1 vecino A

Predicción: A

K = 3



2 B frente a 1 A

Predicción: B

KNN: pros y contras

Qué aporta y qué exige el método

Ventajas	Limitaciones
Simplicidad. Votar entre vecinos resulta fácil de implementar y explicar.	**Coste.** Guarda casos y busca vecinos al predecir. Los árboles no siempre aceleran la consulta.
Flexibilidad. Captura fronteras no lineales sin fijar una distribución paramétrica.	**Escala y distancia.** Las unidades y la métrica pueden cambiar los vecinos y la clase predicha.
Adaptabilidad. Añadir o retirar casos cambia la regla. Puede exigir reconstruir el índice.	**Ruido y dimensión.** Variables irrelevantes distorsionan la cercanía. Más dimensiones complican la búsqueda.
Conserva los casos. Permite inspeccionar los ejemplos que respaldan una predicción.	**Interpretación.** No selecciona variables por sí solo ni estima efectos causales.
Multiclase. La misma regla de voto admite dos o más categorías.	**Sensible a la medida de similitud.** En función de la distancia los resultados pueden cambiar mucho.

Distancia entre observaciones

Escalas comparables antes de buscar vecinos

Primero estandarizamos cada variable y después calculamos la distancia. Ajustamos la media y la desviación solo con el set de entrenamiento.

Caso	Edad	Ingreso €/mes
A	20	1000
B	35	1300
Nuevo	34	1050

1. ESTANDARIZAR CADA VARIABLE

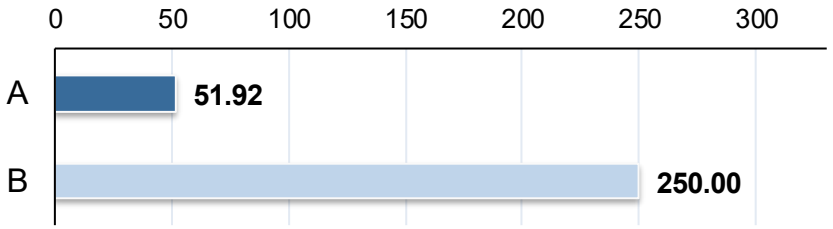
$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j}$$

2. DISTANCIA EUCLÍDEA

$$d(z_i, z_l) = \sqrt{\sum_{j=1}^{p} (z_{ij} - z_{lj})^2}$$

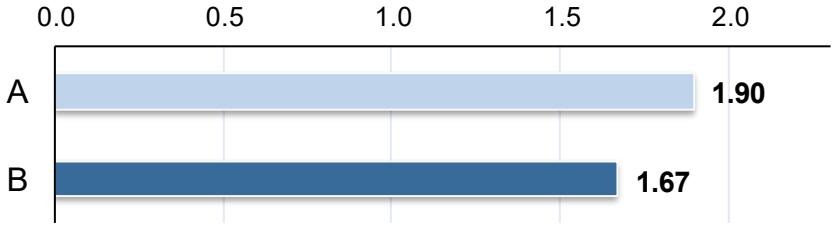
i y l: observaciones. j: predictor. p: nº de predictores. μ_j y s_j se estiman con entrenamiento. Al estandarizar se eliminan las unidades.

Sin escalar: domina el ingreso



Vecino: A

Estandarizado: cambia el vecino



Vecino: B

KNN: algoritmo de clasificación

Datos de partida y pasos para cada caso nuevo

0. $D = \{(x_1, c_1), \ldots, (x_n, c_n)\}$
   Casos con clase conocida.

   $F = \{x_{n+1}, \ldots, x_m\}$
   Nuevos casos por clasificar.

Para cada x_i de F, desde $i = n + 1$ hasta $i = m$:

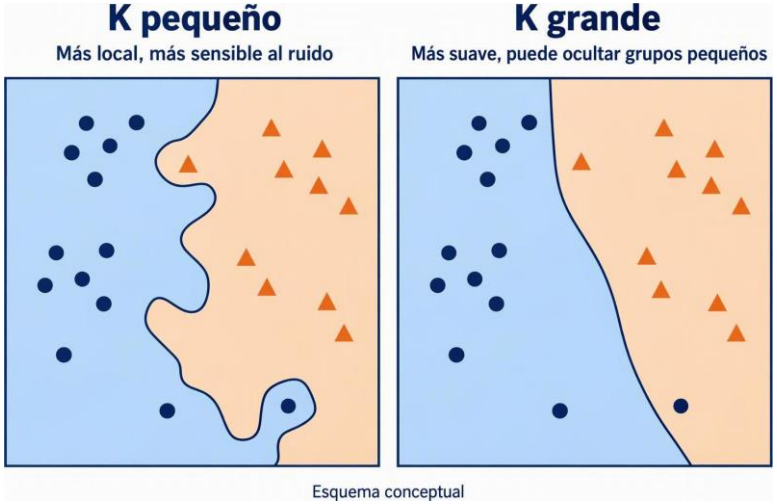
1. **Calcular** $d_j = d(x_i, x_j)$ para cada caso (x_j, c_j) de D.
2. **Ordenar** las distancias d_j de menor a mayor.
3. **Seleccionar** los K casos de D más cercanos a x_i.
4. **Asignar** a x_i la clase más frecuente entre esos K vecinos.
5. Opcional: añadir el caso a D solo cuando conocemos su clase real, no usando la etiqueta predicha como si fuera verdadera.

Fijar K, escala y distancia con entrenamiento. La euclídea es habitual. Prever los empates.

Elección de K

Compromiso entre patrones locales y estabilidad

No hay un K universal ni un rango 3–7 óptimo. Un K impar no evita todos los empates.



Esquema conceptual

K pequeño	K grande
Fronteras más locales	Fronteras más suaves
Menor sesgo, más variabilidad	Mayor sesgo, menos variabilidad
Ruido y casos aislados	Puede ocultar minorías

1. Preparar

Reservar test. Usar métrica y folds de CV comunes para todos los K.

2. Probar

Comparar K y voto con cada parte de entrenamiento en CV y calcular error.

3. Elegir

Elegir el menor error medio. Si gana un extremo, ampliar el rango.

4. Evaluar

Reajustar con todo el entrenamiento y evaluar el test una sola vez.

K vecinos más próximos (KNN)

Python: ejecución y resultados

Implementación

```
knn = make_pipeline(
          StandardScaler(),
          KNeighborsClassifier()
)
param_grid = {
    "kneighborsclassifier__n_neighbors": [3, 7, 15]
}
busqueda = GridSearchCV(knn, param_grid,
    scoring="balanced_accuracy", cv=cv)
busqueda.fit(X_train, y_train)
knn_final = busqueda.best_estimator_
print(busqueda.best_params_, busqueda.best_score_)

for k, score in zip(busqueda.cv_results_["params"],
    busqueda.cv_results_["mean_test_score"]):
    print(k, score)
```

Validación cruzada

Balanced accuracy media

K = 3	**97,26 %**
K = 7	96,31 %
K = 15	96,43 %

Elegido

K=3

Mismo resultado con voto por distancia en los tres K.

Variantes de KNN

Rechazo, distancias y ponderación

KNN con rechazo	Sólo se asigna a una clase si un % de los K vecinos próximos pertenece a esa clase.
KNN con distancia media	Asigna la clase con menor distancia media al caso nuevo. Hay que fijar qué casos se promedian.
KNN con distancia mínima	Usa un representante por clase, como su baricentro, y elige el más cercano. Reduce el coste.
KNN ponderando por los casos	Asigna un peso a cada caso y suma los pesos por clase entre los K vecinos. Gana la suma mayor.
KNN ponderando por las variables	Asigna pesos a las variables al calcular la distancia. Los pesos cambian qué vecinos resultan cercanos.

Con baricentros, la regla equivale al centroide más cercano. Validar las reglas y los pesos.

Variables irrelevantes

Más variables no siempre significan mejores vecinos

Problema: variables sin relación con la clase pueden distorsionar la cercanía y empeorar la predicción.

Selección

Retirar predictores sin utilidad o redundantes y comprobar el efecto con validación cruzada.

Ponderación

Tras escalar, reducir el peso de variables poco útiles. Ajustar los pesos para minimizar el error de CV.

PCA

Reduce dimensiones conservando variación. Puede perder señal de clase y también requiere validación.



No hay un límite universal de variables. Selección y pesos se ajustan dentro de cada fold, con el test reservado.

KNN: la distancia define los vecinos

La fórmula del kernel, término a término

En KNN: los vecinos dependen de la distancia. x y z son dos casos. φ(x) y φ(z) son sus versiones transformadas. El kernel calcula su cercanía sin construir esas versiones.

$$d_\varphi(x,z)^2 = K(x,x) + K(z,z) - 2K(x,z)$$

$$K(x,z) = \langle \varphi(x), \varphi(z) \rangle = \varphi(x)^{\mathrm{T}} \varphi(z)$$
$$K(x,x) = \|\varphi(x)\|^2 \qquad K(z,z) = \|\varphi(z)\|^2$$

Cálculo: $\langle \cdot, \cdot \rangle$ es el producto escalar y $\|\cdot\|^2$ el tamaño al cuadrado. La fórmula combina estos términos: menor distancia, mayor cercanía.

Clasificación

Naive Bayes

03

Naive Bayes

Introducción

Es un clasificador supervisado que asigna cada caso a una de las clases.

Premisa: supone independencia entre predictores dentro de cada clase.

$$P(C \mid x) = \frac{f(x \mid C)\, P(C)}{f(x)}$$

Teorema de Bayes: verosimilitud × probabilidad previa / evidencia

Aplicación: C es una clase y x los predictores. Calculamos P(C | x) para cada clase y elegimos la más probable.

Estimación: usamos el entrenamiento, sin consultar el test. Para predictores continuos, f representa una densidad.

Naive Bayes: pros y contras

Ventajas de la simplicidad y límites del supuesto

Ventajas	Limitaciones
Simplicidad. Combina la probabilidad previa y la aportación de cada predictor para decidir la clase.	**Independencia.** Puede contar dos veces evidencia redundante y fallar ante dependencias importantes.
Rapidez. Para una clase, el coste crece linealmente con los predictores. El ajuste habitual no es iterativo.	**Probabilidades.** Puede clasificar bien y producir probabilidades demasiado extremas. Conviene calibrarlas.
Escalabilidad. Guarda parámetros, no todos los casos. Admite muchos predictores y modelos compactos.	**Ceros.** Una categoría ausente puede anular el producto. El suavizado, como Laplace, lo evita.
Pocos parámetros. Puede estimarse con menos datos que una distribución conjunta completa.	**Probabilidades previas.** Un muestreo sesgado puede alterar las frecuencias de clase y las predicciones.
Referencia útil. Es una primera opción rápida, con buen rendimiento en tareas como la clasificación de textos.	**Distribución elegida.** Cada variante debe encajar con los datos. GaussianNB supone normalidad por clase.

Naive Bayes

Algoritmo

- Se calcularán las probabilidades a posteriori de cada clase para cada individuo con la fórmula de Bayes asumiendo independencia entre las características dentro de cada clase:

$$P(Clase\ 1 \mid Individuo\ 1) = 0.01$$
$$P(Clase\ 2 \mid Individuo\ 1) = 0.1$$
$$\dots$$
$$P(Clase\ k \mid Individuo\ 1) = 0.2$$

- Se asignará el individuo a la categoría con mayor probabilidad (normalmente se omite el denominador porque es igual para un mismo individuo).

$$Individuo\ 1 \rightarrow Clase\ k$$

- Se puede calcular el **Factor de Bayes (FB)** entre dos categorías, si sus probabilidades a priori son iguales, que nos dice cuánto más probable es una categoría respecto a otra.

$$FB = \frac{P(Clase\ k \mid Individuo\ 1)}{P(Clase\ 2 \mid Individuo\ 1)} = \frac{0.2}{0.1} = 2$$

- El individuo 1 tiene el doble de posibilidades de pertenecer a la categoría k que a la categoría 2.

Naive Bayes: ejemplo de clasificación

¿Comprará una mujer de 30 años?

Género	Edad	Compra
Hombre	40	Sí
Hombre	42	Sí
Hombre	72	Sí
Mujer	53	Sí
Hombre	41	No
Mujer	73	No
Mujer	56	No
Mujer	64	No
Mujer	43	No
Mujer	23	No

1. Estimación con los 10 casos

P(Sí) = 4/10 P(Mujer | Sí) = 1/4

P(No) = 6/10 P(Mujer | No) = 5/6

Edad normal por clase (media μ y desviación σ)

Sí: μ = 51,75 y σ = 12,70. f(30 | Sí) ≈ 0,00724

No: μ = 50 y σ = 16,43. f(30 | No) ≈ 0,01158

2. Puntuación = previa × género × densidad

Sí: 0,4 × 0,25 × 0,00724 ≈ 0,000724

No: 0,6 × (5/6) × 0,01158 ≈ 0,005788

3. Normalización y decisión

P(No | Mujer, 30) ≈ 0,005788 / 0,006512 ≈ 88,9 %

Predicción: «No comprar». Razón No/Sí ≈ 8.

Probabilidades nulas y estabilidad

Laplace y var_smoothing resuelven problemas distintos

En datos categóricos, una categoría que no aparece en una clase puede hacer cero todo el producto. Laplace evita ese problema añadiendo pseudoconteos.

$$P(x_j = v \mid c) = \frac{N_{jvc} + \alpha}{N_c + \alpha V}$$

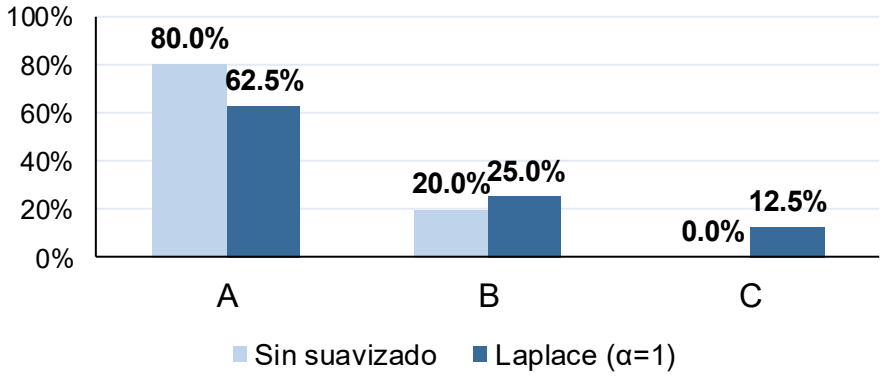
N_{jvc} cuenta las veces que aparece una categoría de la variable j en la clase c.

Nc cuenta las observaciones de esa clase.

V cuenta las categorías posibles de la variable y α controla los pseudoconteos (normalmente vale 1).

Ejemplo ilustrativo dentro de una clase

Conteos por categoría: A=4, B=1, C=0



La categoría C pasa de 0 % a 12,5 %

Con α=1: C recibe (0+1)/(5+3)=1/8.

Se añade un pseudoconteo a cada categoría.

Naive Bayes: tasa de error

Error teórico y error observado

Elegimos la clase con mayor probabilidad posterior. Con probabilidades reales, el error mínimo para un caso x es:

$$e^*(x) = 1 - \max_j P(Y = j \mid X = x)$$

Ejemplo ilustrativo: una probabilidad máxima de 0,80 deja un 20 % de riesgo de error.

Error de Bayes: promedio sobre X y mínimo teórico con esos predictores.

$$R^* = 1 - E[\max_j P(Y = j \mid X)]$$

Tasa de error observada

Naive Bayes estima las probabilidades.

Proporción de casos mal clasificados en evaluación:

1 − accuracy

Naive Bayes

Python: ajuste y probabilidades con GaussianNB

Misma partición y mismos folds para comparar.

Implementación

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import cross_val_score

nb = make_pipeline(
    SimpleImputer(strategy="median"),
    GaussianNB(var_smoothing=1e-9)
)
scores_nb = cross_val_score(
    nb, X_train, y_train,
    cv=cv, scoring="balanced_accuracy"
)
nb.fit(X_train, y_train)
prob_nb = nb.predict_proba(X_test)
```

Probabilidades

`prob_nb[1]`



Clase 0: ≈0,0000

Clase 1: 0,6391

Clase 2: 0,3609

Predicción: clase 1

Media CV

97,26 %

KNN y Naive Bayes: criterios de elección

Ventajas habituales y límites de cada método

Criterio	KNN	Naive Bayes	Observaciones
Tiempo computacional	✗	✓	NB predice rápido. KNN busca vecinos entre los datos.
Número de variables	✗	✓	En KNN, muchas variables degradan las distancias.
Independencia	✓	✗	NB exige independencia condicional. KNN no la exige.
Interacciones	✓	✗	NB no modela interacciones. KNN puede captarlas.
Sesgo	±	±	Depende de K en KNN y de los supuestos en NB.
Varianza	±	±	Con K pequeño, KNN suele tener mayor varianza.
Error medio	±	±	Ninguno garantiza menor error. Decide la validación.
Regiones de decisión	✓	±	KNN admite fronteras flexibles. En NB dependen del modelo.
Atributos irrelevantes	✗	±	Las variables irrelevantes pueden perjudicar a ambos.
Interpretabilidad	±	±	NB explica aportes por variable. KNN usa vecinos.
Parámetros	±	✓	KNN ajusta K y la métrica. NB tiene hiperparámetros.
Datos ausentes	✗	✗	KNN y GaussianNB requieren imputación en scikit-learn.

✓ Ventaja habitual ✗ Limitación ± Depende del caso. La validación decide.

Clasificación

Ensambles

04

Bagging y boosting

Definición y consideraciones

Bagging

Genera muestras bootstrap con reemplazamiento: unos casos se repiten y otros quedan fuera. Ajusta modelos en paralelo y combina votos o probabilidades. Busca reducir varianza: que la predicción dependa menos de la muestra.

Boosting

Entrena modelos sencillos en secuencia. Cada modelo intenta corregir errores del conjunto anterior. Combina sus aportaciones con pesos. Busca reducir sesgo: el error de un modelo demasiado simple.

Sobreajuste

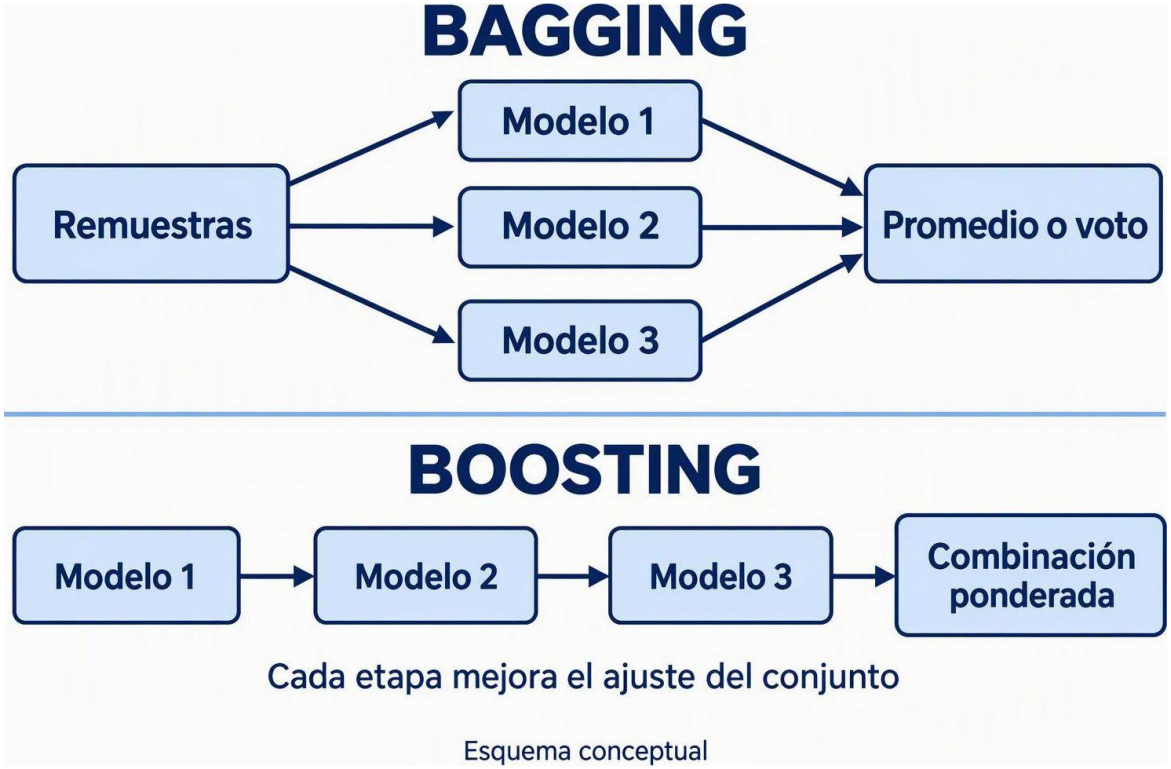
Ambos pueden mejorar la generalización, pero también sobreajustar. En boosting hay que controlar las etapas, la tasa de aprendizaje y la complejidad.

Cuándo ayudan

Bagging con modelos inestables. Boosting con modelos demasiado simples.

Bagging y boosting

Principios y límites de los ensambles



Bagging reduce variabilidad. Random forest añade selección aleatoria de predictores.

Boosting controla el ajuste con la tasa de aprendizaje y la complejidad.

Ambos pueden sobreajustar. La comparación usa las mismas particiones y métrica.

Recordatorio

Validación y métricas

05

Recordatorio: clase, probabilidad y umbral

Cada salida requiere una evaluación diferente

Pregunta	Medida o comprobación
¿Qué errores comete la etiqueta?	Matriz de confusión, sensibilidad y especificidad
¿Rinde en todas las clases?	Sensibilidad por clase y promedio macro
¿Las probabilidades son fiables?	Log-loss y curva de calibración
¿Qué umbral conviene?	Costes de error y validación separada

Sensibilidad: proporción de positivos predichos que son correctos.

Recall/Especifcidad: proporción de negativos reales que el modelo detecta.

Recordatorio: medidas de rendimiento

Una métrica responde a una pregunta concreta

ROC AUC: ¿ordenamos los positivos por encima de los negativos? 0,5 indica una ordenación sin información.

Average precision (AP): resume precisión y recall al variar el umbral. Es útil cuando la clase positiva es poco frecuente.

Brier: error cuadrático de las probabilidades.

$$Brier = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)^2$$

Log-loss: penaliza las probabilidades erróneas muy seguras.

$$Log - loss = -\frac{1}{n}\sum_{i=1}^{n}[y_i \, log(p_i) + (1 - y_i) \, log(1 - p_i)]$$

y_i indica la clase (0/1), p_i su probabilidad predicha y n el número de observaciones. Menor Brier o log-loss es mejor. Accuracy puede ocultar positivos no detectados.

Validación y métricas

Python: evaluación final del modelo elegido

El modelo y la métrica quedan fijados antes de abrir el test.

Implementación

```
from sklearn.metrics import (
    classification_report, log_loss,
    confusion_matrix)

# Modelo elegido con entrenamiento
modelo = knn_final
pred = modelo.predict(X_test)
prob = modelo.predict_proba(X_test)

print(
    log_loss(
      y_test, prob, labels=modelo.classes_
    )
)
print(confusion_matrix(y_test, pred))
```

Matriz de confusión

Filas: real. Columnas: predicha.

Real / Pred.	0	1	2
0	**12**	0	0
1	0	**13**	0
2	0	**3**	**10**

3 errores: clase 2 como clase 1

Accuracy 92,11 %

Log-loss 1,0383

El test informa del rendimiento. No reajustar el modelo ni el umbral con él.

Salida calculada con Iris. Demostración reproducible, random_state=42.

MUBD

Màster Universitari en Enginyeria de Dades Massives (Big Data)

Estadística

laSalle ENG
Universitat Ramon Llull